Minigraf 0507

control with Arduino Nano Every

from an Atari XE/XL computer (SIO) and from a PC (USB)



By Zdeněk Kotrba

© 2026 · version 0.3.8 (firmware 0.3.8)

1. What the interface does

The Aritma Minigraf A 0507 is a Czechoslovak pen plotter from the 1980s. It has no intelligence of its own: it only accepts a phase number for each stepper motor (X1, X2, X4 and Y1, Y2, Y4) and a pen signal. The Arduino interface does this work for the computer – it accepts readable commands (simplified HPGL), converts lines into motor steps and handles acceleration and deceleration.

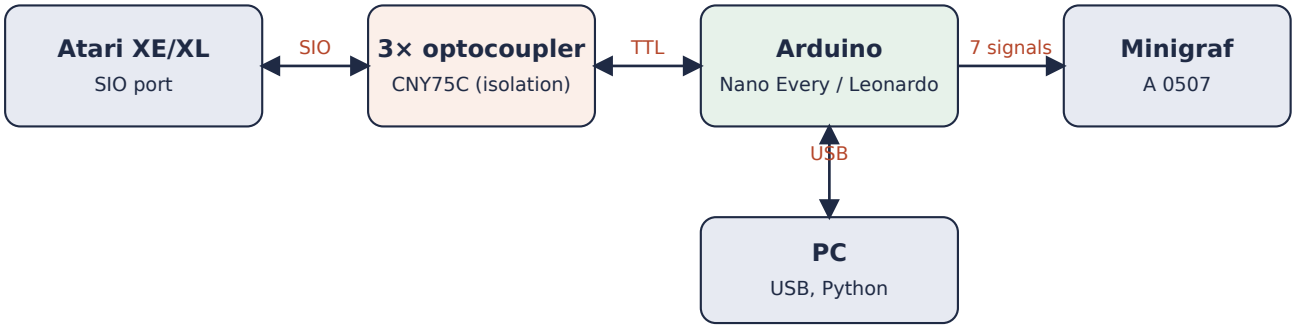


Fig. 1 – Overall setup

- **From the Atari** you draw with a plain `LPRINT` – the Arduino appears on the SIO bus as printer **P1:**.
- **From a PC** you draw with Python programs (`plot.py`, `text_plot.py`) or directly from the Arduino IDE Serial Monitor.
- The Atari is **galvanically isolated** by three optocouplers; the plotter connects to the Arduino directly.
- Transfer uses **handshaking**: the next command is sent only after the plotter has finished the previous one. Nothing gets lost.

2. Parts list

Part	Qty	Note
Arduino Nano Every (ATmega4809)	1	recommended; alternative Arduino Leonardo / Micro (chapter 3)
Optocoupler CNY75C (or CNY75A/B)	3	OK1 Data Out, OK2 Command, OK3 Data In
Resistor 1 kΩ	3	optocoupler LED current
Resistor 2.2 kΩ	2	pull-up on OK1 and OK2 outputs
Resistor 470 kΩ	3	base–emitter, speeds up transistor turn-off
SIO connector, 13 pin (or a cable from an SIO device)	1	pins 3, 4, 5, 7, 10
Minigraf connector, 12 pin, 2.54 mm pitch	1	key at position 7, bottom row of contacts only
Optional: 7× 220 Ω, 100 µF + 100 nF	–	signal protection, Arduino supply decoupling

Power

- **Arduino** is powered from USB (PC or USB adapter). Do not power it from the plotter.
- **Minigraf** has its own **24 V** supply (about 30 W, supply rated at least 1.5 A). Supply minus is also the logic ground – pins 1 and 11 of the data connector.
- **The Atari side** (optocoupler LEDs) is powered from SIO pin 10 (+5 V).

Important: Atari ground (GND_A) and Arduino/plotter ground (GND_D) must **never be connected**. That is what the optocouplers are for.

3. Supported Arduino boards

The firmware `minigraf_sio.ino` detects the board from the processor. Both boards have a separate hardware UART (`Serial1` on D0/D1) for SIO, so USB stays free for the PC and debugging. All six motor phases are always on one processor port and change at the same instant.

Signal	Nano Every	Leonardo / Micro	Minigraf pin	Function
SIO Data Out (via OK1)	`D0 (RX1)`	`D0 (RX1)`	–	data from Atari
SIO Data In (via OK3)	`D1 (TX1)`	`D1 (TX1)`	–	replies to Atari
SIO Command (via OK2)	`D2`	`D2`	–	start of SIO command
X1	`A0`	`A0`	`10`	X phase, bit 0
X2	`A1`	`A1`	`9`	X phase, bit 1
X4	`A2`	`A2`	`8`	X phase, bit 2
Y1	`A3`	`A3`	`4`	Y phase, bit 0
Y2	`A6`	`A4`	`3`	Y phase, bit 1
Y4	`A7`	`A5`	`2`	Y phase, bit 2
P (pen)	`D10`	`D10`	`5`	H = pen down
GND	`GND`	`GND`	`1, 11`	common ground with plotter

	Nano Every (recommended)	Leonardo / Micro
Processor	ATmega4809, 48 kB flash, 6 kB RAM	ATmega32U4, 32 kB flash, 2.5 kB RAM
Board package in Arduino IDE	Arduino megaAVR Boards (clones: MegaCoreX, Pinout "Nano Every")	Arduino AVR Boards (built in)
Motor phase port	PORTD (A0–A3, A6, A7)	PORTF (A0–A5)
Note	A6/A7 are full digital pins on the Nano Every	Micro uses the same pin labels

Minigraf connector

The cable connects to the plotter with the same type of edge connector used by ZX Spectrum bus peripherals, with **2.54 mm** pitch. It has only **12 positions**, and **position 7 is a key**. The plotter PCB has contacts on the bottom side only, so **only the bottom row** of the connector is used.



Fig. – Connector position numbering (key at position 7 shown in black)



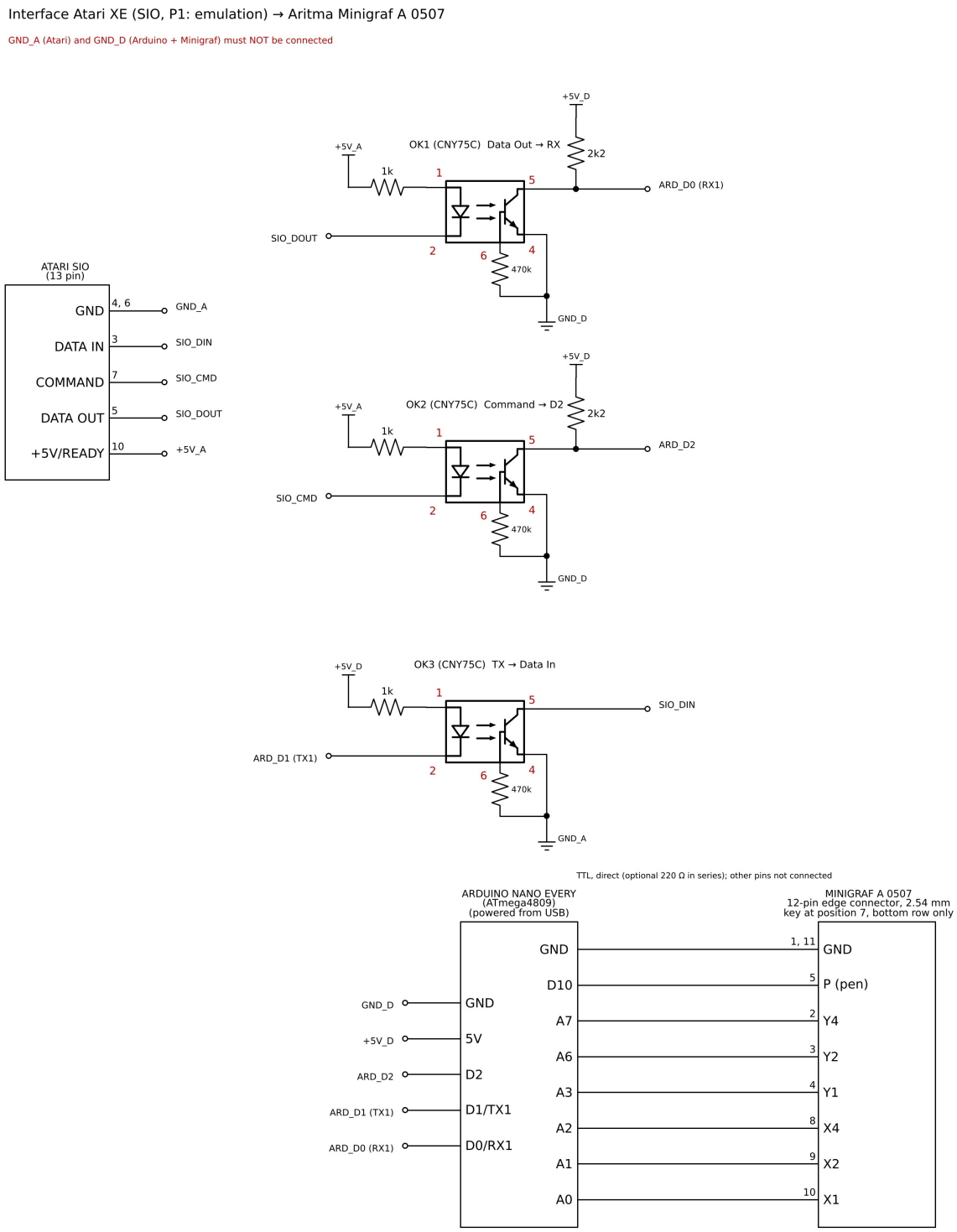
	12	11	10	9	8	7	6	5	4	3	2	1
signal:	**nc**	**GND**	**X1**	**X2**	**X4**	*key*	**nc**	**P**	**Y1**	**Y2**	**Y4**	**GND**
Nano Every:		GND	A0	A1	A2			D10	A3	A6	A7	GND

bottom row of plotter contacts (position 7 = key, no contact)

Fig. – Signals at each position and the matching Arduino Nano Every pins

Caution: the key **counts as pin 7** in the numbering even though it has no contact. Contacts to the left of the key are therefore 8 to 12, not 7 to 11. Numbers moulded on some connectors do not match this numbering – check with an ohmmeter that pins 1 and 11 are plotter ground before connecting.

4. Schematic

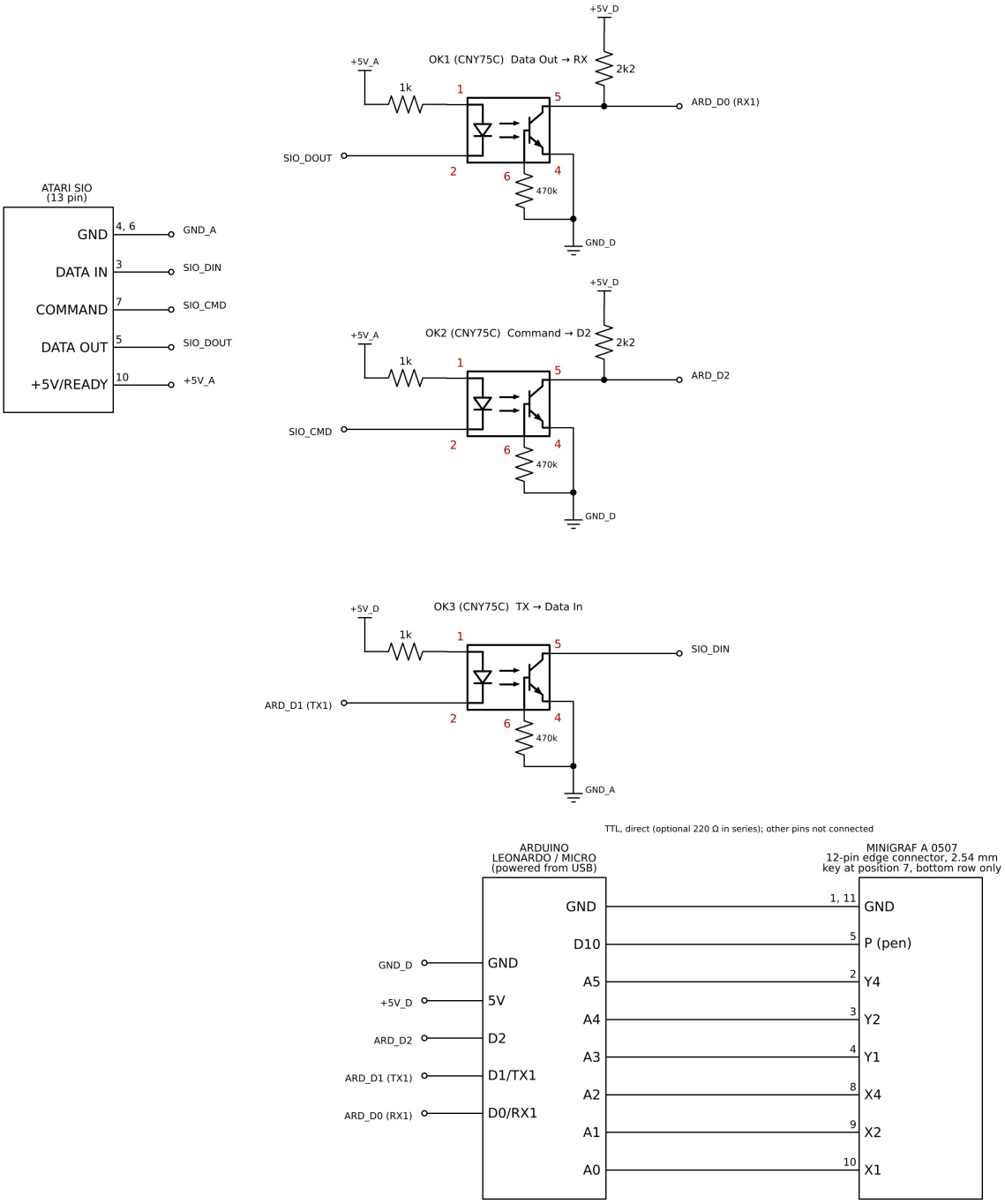


Note: +5V_A = SIO pin 10 (Atari), +5V_D = Arduino 5 V. All optocouplers are non-inverting.
If D0 is not below 0.8 V while the Atari transmits, reduce 1k at OK1/OK2 to 680 Ω.

Fig. 2 – Wiring with Arduino Nano Every

Interface Atari XE (SIO, P1: emulation) → Aritma Minigraf A 0507

GND_A (Atari) and GND_D (Arduino + Minigraf) must NOT be connected



Note: +5V_A = SIO pin 10 (Atari), +5V_D = Arduino 5 V. All optocouplers are non-inverting.
If D0 is not below 0.8 V while the Atari transmits, reduce 1k at OK1/OK2 to 680 Ω.

Fig. 3 – Alternative wiring with Arduino Leonardo / Micro (only Y2 = A4 and Y4 = A5 differ)

5. Software installation

Arduino firmware

- In the Arduino IDE open the **Boards Manager**, search for **megaAVR** and install **Arduino megaAVR Boards**.
- Select **Tools → Board → Arduino megaAVR Boards → Arduino Nano Every** and the correct port.
- Open `minigraf_sio.ino` and upload it.
- The message `jtagmkII_initialize(): Cannot locate "flash" and "boot" memories` is normal and harmless on the Nano Every.
- Nano Every clone with a CH340 USB chip: use **MegaCoreX** instead, board ATmega4809 and **Pinout: Nano Every**.

Python on the PC (Windows)

- Install Python 3 and run in the **command prompt** (not in the Python window with >>>):

```
py -m pip install pyserial
py -m pip install matplotlib      (only for --preview)
```

- Put `plot.py` and `text_plot.py` in the same folder, e.g. `C:\plotter`.
- The default port is **COM4**. Give another port before the file name or with `--port`.
- The **Serial Monitor must be closed** while plotting from Python – only one program can open the port.

6. Coordinates and plotting area

Parameter	Value
Unit	1 step = 0.125 mm (8 steps = 1 mm)
Absolute X position	0 = left carriage stop (after HM), limit 1580 steps (verified, changed by LM)
Absolute Y position	0 = paper loaded with its top edge at the cut-outs under the feed wheels, limit 2284 steps (verified)
Start position	X = 60, Y = 144 – the head moves here after HM
Drawing area from start	X 0 – 1520 steps (190 mm), Y 0 – 2140 steps (267.5 mm)
A4 margins (measured)	left and right 10 mm, top and bottom 15 mm
Drawing point 0,0	set by IN; – the PC programs send HM;IN; automatically
Centre of area (after HM;IN;)	X = 760, Y = 1070

Homing – HM (from version 0.3)

The plotter has no limit switch, but the carriage has a mechanical stop. `HM;` lifts the pen and drives the head left by the full width plus 200 extra steps. At the stop the stepper simply slips (harmless to the plotter), so the head always ends up exactly at the left, wherever it was. The firmware sets X = 0 there and moves to the start position. The Y axis (paper) has no stop: the paper position at the moment of HM is Y = 0, so A4 paper is **always loaded with its top edge at the cut-outs under the feed pinch wheels**, and HM is sent right after. The start position X = 60, Y = 144 gives A4 margins of 10 mm at the sides and 15 mm at the top. The Y limit 2284 leaves 15 mm at the bottom too – so the paper does not slip out of the feed wheels (with a longer area it slipped out by about 3 mm).

From then on the firmware knows the **absolute position** from the stop and checks the area limits against it – `IN;` only shifts the drawing origin, the limits stay in place. Limits are set with `LM x,y;` and stored in EEPROM, so they survive power-off.

The PC programs send `HM;` automatically before every drawing (disable with `--nohome`). From the Atari, start your program with `LPRINT "HM;IN;"`.

7. Command reference

Commands are the same from the Atari, the Serial Monitor and in PC files. Each command starts with two letters, numbers are separated by commas and commands by semicolons. Several commands may be on one line.

Command	Meaning	Example
`IN;`	initialise: current position = 0,0, pen up, absolute coordinates	`IN;`
`PU x,y;`	pen up and move to x,y (several points allowed)	`PU760,1070;`
`PD x,y,...;`	pen down and draw through the given points	`PD800,100,800,900;`
`PA;`	following coordinates are absolute (default)	`PA;PU0,0;`
`PR dx,dy;`	following coordinates are relative to current position	`PR80,0;`
`CI r[,n];`	circle of radius r around current position, n segments (default 72)	`PU750,1050;CI400;`
`VS v;`	speed in mm/s; VS; = default (60 mm/s)	`VS40;`
`PT d,u;`	pen delay in ms after pen down / pen up; PT; = default (30 / 70)	`PT20,70;`
`HM[x[,y]];`	home to left stop (X = 0), paper = Y 0, then move to start (default 60,144)	`HM;`
`LM x[,y];`	area limits in steps from the stop, stored in EEPROM; LM; = print (default 1580,2284)	`LM1580,2284;`
`AC s,r;`	acceleration: start delay s µs, shortened by r µs per step; AC; = default (6000 / 250)	`AC5000,300;`
`DG0; / DG1;`	diagonal lines at full speed / slowed ×1.4; DG; = default (0)	`DG1;`
`?`	Serial Monitor only: firmware version and status (position from origin and absolute, limits, pen, speed, AC, DG, PT)	`?`

Useful moves

What	Command
Home to left stop and start position	`HM;`
Head back to drawing origin	`PU0,0;`
Head to centre of area (after HM;IN;)	`PU760,1070;`
Move 10 mm right (without drawing)	`PU;PR80,0;PA;`
Square 50 × 50 mm	`PU200,200;PD600,200,600,600,200,600,200,200;PU;`
New origin at current position	`IN;`

Tuning acceleration (from version 0.2)

Every line starts slowly, accelerates to the set speed and slows down before its end. From version 0.3.2 the default is the verified setting `AC6000,250` (full speed after 2 mm) and diagonal lines run at full speed (`DG0`). For a gentler setting on an older or worn plotter use `AC9000,150;DG1;`. You can detect lost steps when the head does not return exactly to the origin after `PU0,0;`.

Setting	Start	Full speed after	Note
`AC9000,150;`	14 mm/s	46 steps (5.8 mm)	gentlest (default up to 0.3.1)
`AC6000,250; (default)`	21 mm/s	16 steps (2 mm)	verified: print looks good
`AC5000,300;`	25 mm/s	10 steps (1.2 mm)	fast; check return to origin

Firmware defaults

Constant in minigraf_sio.ino	Value	Meaning
STEP_US	2083	speed 60 mm/s (= 125000 / speed)
START_US / RAMP_US	6000 / 250	start and stop of every line (changed by AC, verified)
DIAG_SLOW	false	diagonal lines at full speed (changed by DG, verified)
FW_VERSION	"0.3.8"	firmware version, printed at start-up and by ?
X_MAX_DEF / Y_MAX_DEF	1580 / 2284	default area limits from the stop (overridden by LM stored in EEPROM)
HOME_X / HOME_Y	60 / 144	start position after HM (A4: 10 mm left margin, 15 mm top margin)
HOME_DIR	−1	direction to the left stop (+1 = opposite)
HOME_EXTRA / HOME_US	200 / 2500	extra steps beyond the width / homing speed (~50 mm/s)
MIN_STEP_US	1500	maximum allowed speed about 83 mm/s
PEN_DOWN_MS / PEN_UP_MS	30 / 70	pen delay (verified with a 0.5 mm marker)
PEN_IDLE_MS	1000	pen down and no command for this long → firmware lifts it (no ink blot on a dropout)
PEN_DOWN_LEVEL	HIGH	P signal level for pen down
X_DIR / Y_DIR	+1 / +1	change to −1 to reverse an axis
PRN_TIMEOUT_S	255	how long the Atari waits for a line to finish

8. Plotting from the Atari

On the SIO bus the Arduino behaves as printer **P1:**. You draw with a plain LPRINT in Atari BASIC (or by writing to device P: from other programs). After each line the Atari waits until the Arduino confirms the drawing is finished – no timing needed.

Example Atari BASIC program

```
10 REM MINIGRAF 0507 - DEMO
20 LPRINT "HM;IN;"
30 REM FRAME
40 LPRINT "PU100,100;PD1400,100,1400,2000,100,2000,100,100;"
50 REM CIRCLE IN THE MIDDLE
60 LPRINT "PU750,1050;CI400;"
70 REM VERTICAL LINES
80 FOR I=0 TO 20
90 LPRINT "PU";100+I*60;",300;PD";100+I*60;",900;"
100 NEXT I
110 LPRINT "PU0,0;"
```

Quick commands from the Atari

Atari BASIC	Result
LPRINT "HM;IN;"	home to the stop, start position = origin
LPRINT "PU0,0;"	head back to drawing origin
LPRINT "VS40;"	slower drawing (40 mm/s)
LPRINT "VS;PT;"	back to default speed and pen delay

Atari BASIC	Result
`LPRINT "AC6000,250;DG0;"`	faster start and diagonal lines at full speed
`LPRINT "PU750,1050;CI200;"`	circle 50 mm in diameter in the middle

Atari tips

- One `LPRINT` may contain several commands; longer lines are split into several SIO records automatically.
- Atari BASIC prints numbers without spaces, so `LPRINT "PU";X;",";Y;";"` works directly.
- If a single line took longer than about 4 minutes, the Atari would report error 138 – split such a line into several `LPRINT`.
- While commissioning, the Serial Monitor shows `SIO: status` and `SIO: zapis OK` for each `LPRINT`.

9. Plotting from a PC

9.1 Serial Monitor (manual commands)

In the Arduino IDE open the Serial Monitor, set **115200 baud** and **Newline**. Each line you type is drawn and the Arduino replies `hotovo` ("done"). Board LED blinking = program running, steady = drawing.

9.2 Plotting an HPGL file – plot.py

```
py plot.py drawing.hpgl              (HPGL from Inkscape, port COM4)
py plot.py napis.hpgl --scale 1      (file already in plotter steps)
py plot.py COM7 drawing.hpgl         (another port)
py plot.py drawing.hpgl --dry        (only print what would be sent)
```

Option	Meaning
`--scale 0.2`	coordinate scale; 0.2 = HPGL from Inkscape (40 units/mm), 1 = plotter steps
`--speed 40`	speed in mm/s for this drawing
`--pen 0,70`	pen delay in ms (after pen down, after pen up)
`--accel 6000,250`	acceleration for this drawing (same as AC command)
`--diag 0`	diagonal lines at full speed (1 = slowed)
`--nohome`	do not home (HM) first; draw from where the head is
`--dry`	send nothing, only print the converted commands
`--version`	print program version; the firmware version is printed automatically after connecting

Plotting from Inkscape: File → Save As → *HP Graphics Language (*.hpgl)*, resolution 1016 dpi, leave "Center zero point" unticked.

9.3 Lettering and text – text_plot.py

The program has its own stroke font (upper and lower case, digits, punctuation, Czech diacritics). Three modes:

```
Large lettering across the whole width (A4 landscape):
  py text_plot.py "HELLO" --frame --star --bold 3 --send

Normal text in lines, wrapped automatically ( | = new line):
  py text_plot.py "Hello,|this is Minigraf." --size 5 --send
  py text_plot.py --file letter.txt --size 4 --portrait --send

Test page (6 / 4 / 3 / 2 mm):
  py text_plot.py --test --portrait --send
```

Option	Meaning
`--send`	send to the plotter right away (otherwise only creates napis.hpgl)
`--size 5`	capital letter height in mm – normal text mode
`--portrait`	A4 portrait like a printer (lines along the short side)
`--file letter.txt`	text from a file (save as UTF-8)
`--frame / --star`	double frame / eight-pointed star above large lettering
`--bold 1–4`	bold: letters are traced several times with a small offset
`--margin 8`	margin inside the drawing area in mm (default 8 for lettering, 5 for text; the area already has A4 margins)

Option	Meaning
`--flip`	rotate the drawing by 180° if it comes out upside down
`--preview`	save a .png preview (needs matplotlib)
`--speed / --pen / --accel / --diag / --nohome / --port`	same as plot.py
`--version`	program version

Recommended letter size by pen

Pen line width	Smallest well-readable letters
0.5 mm	about 5 mm
0.3 mm	about 3 mm
0.1–0.2 mm	about 2 mm

10. Example results



Fig. 4 – Large lettering with frame and star, speed 80 mm/s, 0.5 mm marker.
`py text_plot.py "ZDENĚK 0507" --frame --star --bold 3 --send --speed 80`

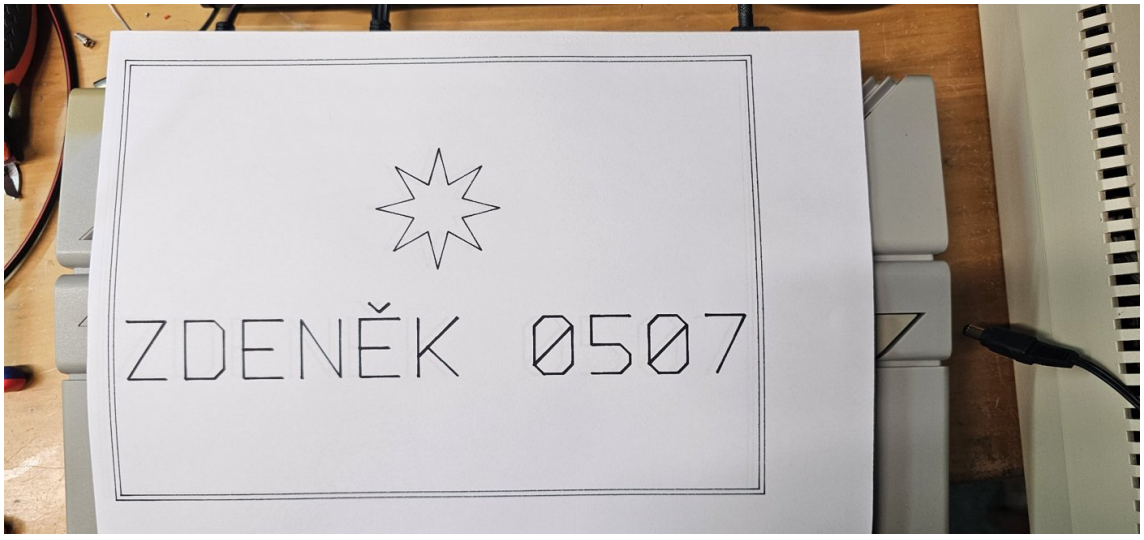
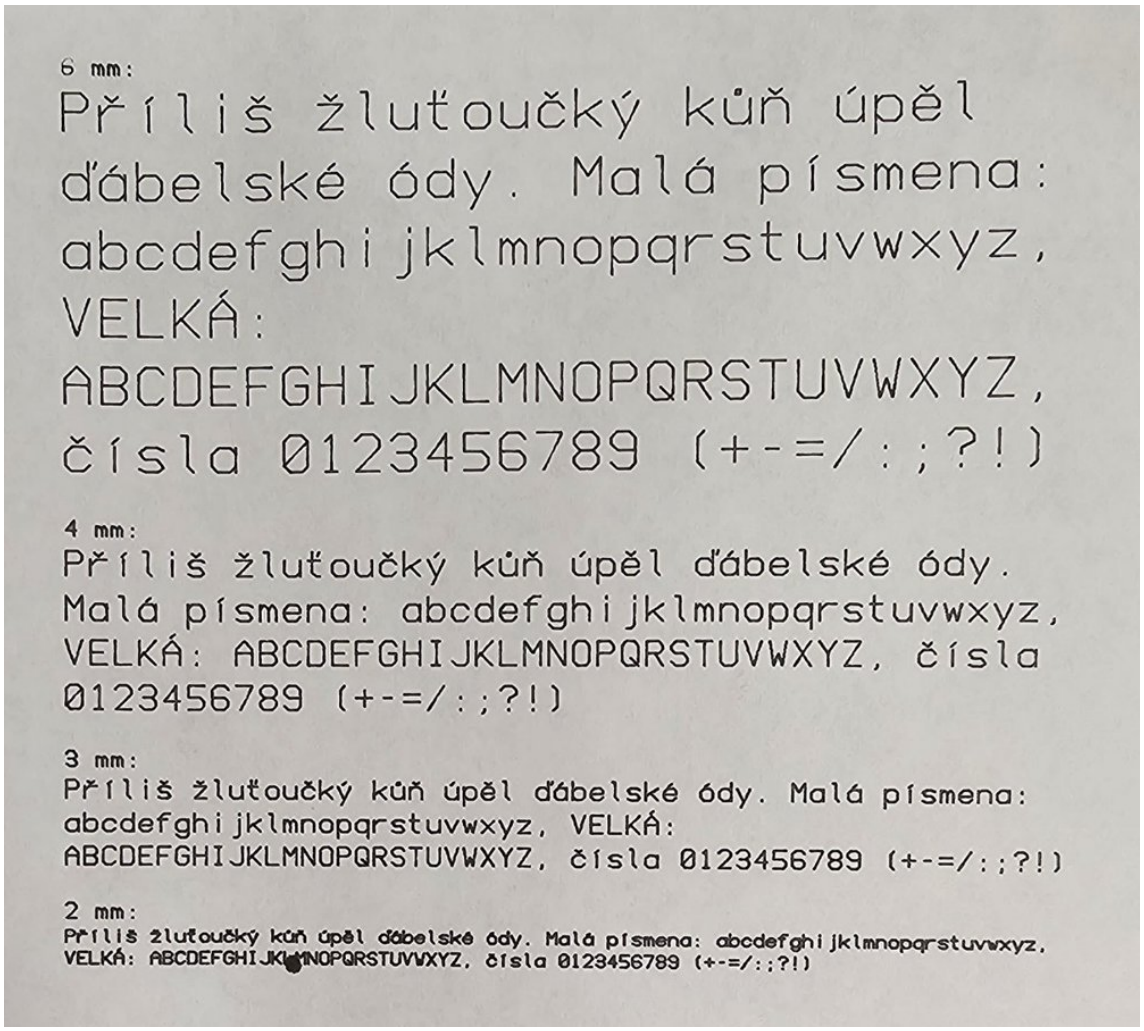


Fig. 5 – The same drawing from the first tests, view of the plotter.

Fig. 6 – Test page

`py text_plot.py --test --portrait --send` – firmware and programs 0.3.4, default settings (speed 60 mm/s, acceleration AC6000,250, diagonal lines DG0, pen delay PT30,70), 0.5 mm marker, 419 s.



- 6, 4 and 3 mm: clean and easy to read, loops close, diacritics in place, lines are straight.
- 2 mm: readable, but a 0.5 mm marker is at its limit – letters look bold. A 0.3 mm marker will be clean.
- **Why the ink blot appeared** (last line near "JKLM"): at line 2058 of 2198 one command was lost on USB (`PD368,836,379,836;`), so the Arduino did not reply "hotovo" (done). Version 0.3.4 waited 20 s before querying the Arduino and resending the command – and all that time the pen stayed down on the paper, so the ink soaked into one spot. The drawing then finished correctly ("Znovu posláno řádků: 1" = 1 line resent).
- **Fix in version 0.3.5:** the firmware lifts the pen by itself when no command arrives for 1 s, and `plot.py` queries after only 3 s. A dropout now delays the drawing by just a few seconds and leaves no blot.

11. Troubleshooting

Symptom	Cause and fix
Head stops and hums	motor is losing steps – lower the speed (--speed or VS), check the 24 V supply and mechanics
plot.py prints "bez odpovědi, posílám znovu" (no reply, resending)	USB communication dropout (up to 0.3.2 mainly an overflow of the 64-byte Arduino buffer, fixed in 0.3.4); plot.py recovers by itself (after 3 s it queries the Arduino and resends the line) and firmware 0.3.5 lifts the pen meanwhile, so no ink blot appears. If frequent: 100 µF + 100 nF on Arduino 5 V, shorter ground to the plotter, short USB cable, ferrite on USB
plot.py reports "ARDUINO SE RESTARTOVALO" (restarted)	the Arduino restarted and the position is lost – run the drawing again (HM homes to the stop); usually caused by interference or a supply dip
could not open port / Access denied	Serial Monitor open or wrong port number
No module named serial	py -m pip install pyserial
Nothing happens from the Serial Monitor	set "Newline" and 115200 baud (bottom right)
Blobs at the start of lines	shorten delay after pen down: --pen 0,70 or PT0,70;
Start of line missing	lengthen delay after pen down: --pen 30,70
Tails or smudges between letters	lengthen delay after pen up: --pen 0,100
Drawing mirrored	in minigraf_sio.ino change X_DIR or Y_DIR to −1
Drawing upside down	option --flip
Pen does the opposite (PD lifts)	change PEN_DOWN_LEVEL to LOW
Nothing after LPRINT, monitor silent	check OK1 and OK2; at rest the status must show Command=H
"chyba souctu" (checksum error) in monitor	slow edges at OK1 – reduce 1 kΩ to 680 Ω
Atari reports error 138	Arduino not responding or one line takes longer than the timeout – split the line
Text lines are slanted	paper loaded skewed – align it with the roller
Marker smears even mid-line	squashed tip – replace the marker
HM moves right instead of left	switch the plotter off at once; set HOME_DIR = +1 in minigraf_sio.ino
Head does not reach the right edge	limit too small: LM; prints it, LM1580,2284; sets it
Paper slips out of the feed wheels	Y limit too large: reduce it, e.g. LM1580,2250;
Drawing is shifted on the paper in Y	paper was not loaded at the cut-outs, or moved after HM – reload it and send HM
Small text is drawn slowly	faster start: --accel 6000,250 or AC6000,250;
plot.py reports firmware "neznámá" (unknown)	Arduino has firmware older than 0.2 – upload the new one
Firmware does not know HM	Arduino has firmware older than 0.3 – upload the new one, or plot with --nohome

12. Project files

File	Content
`firmware/minigraf_sio/minigraf_sio.ino`	firmware for Arduino Nano Every and Leonardo/Micro
`pc/plot.py`	sending HPGL files from a PC
`pc/text_plot.py`	lettering, text in lines and test page
`docs/`	manuals (CZ, EN), schematics and photos

Note: the firmware and PC programs print their messages in Czech. Connector pinout based on cygnus.speccy.cz (Aritma Minigraf 0507); signal meaning from the MZXR ZX Spectrum documentation and the MAME emulator.